

Caso Práctico 2: "CRUD/ORM Nativo en Python sobre la Base de Datos Sakila".

Por
José Miguel Maduro Valenzuela,
José Rafael Berroa Castillo,
Manicaotex Mueses Ruiz

Asignación Presentada a la
Escuela de Informática, Facultad de Ciencias, Como Cumplimiento del Curso "Ciencia
de Datos I" (INF-8237 C2),
Dr. Silverio del Orbe Abad.

Universidad Autónoma de Santo Domingo (UASD)
Mayo 2026

Tabla de Contenido

Resumen	1
Introducción	2
Fase I: CRUD Básico y Consultas SQL	3
Configuración del Entorno.....	3
Consultas SQL (Q1–Q10).....	3
CRUD Básico en Python.....	4
Estructura de las funciones CRUD	4
Importación y exportación CSV / JSON.....	4
Métricas descriptivas	4
Mejora de Integridad: UNIQUE Constraints	5
Fase II: ORM Nativo en POO	5
Arquitectura de la Solución.....	5
DbContext	6
Entities	6
Repositories (List<Entity>).....	7
Controllers (MVC)	7
Flujo Completo de una Operación.....	8
Resultados	8
Estructura del Proyecto	9
Conclusiones y Recomendaciones	9
Referencias	10
Anexo A: Repositorio y Código Fuente	10
Anexo B: Instrucciones de Ejecución	11

Resumen

El presente trabajo documenta el diseño e implementación de un sistema de gestión de datos sobre la base de datos de ejemplo Sakila de MySQL, desarrollado en Python bajo el paradigma de Programación Orientada a Objetos (POO). La solución se estructuró en dos fases complementarias. En la Fase I se realizó la instalación y configuración del entorno MySQL junto con el IDE Workbench, la importación de la base de datos Sakila, la elaboración de diez consultas SQL analíticas sobre entidades clave como películas, ciudades, países y pagos, y el desarrollo de un módulo CRUD básico en Python para las entidades City, Country y Film. Este módulo incorpora funciones de importación y exportación en formatos CSV y JSON, así como el cálculo de métricas descriptivas fundamentales: media aritmética, rango, varianza muestral, desviación estándar y covarianza, implementadas sin dependencias externas. Adicionalmente, se mejora la integridad referencial de la base de datos mediante la adición de restricciones UNIQUE en tablas críticas, acompañadas de diez consultas adicionales de validación. En la Fase II se implementó un ORM nativo de cuatro capas: DbContext (gestión Singleton de la conexión inspirada en el patrón Unit of Work), Entity (espejo de tabla en la base de datos siguiendo el patrón Domain Model), Repository (colección List<Entity> que abstrae el acceso a datos) y Controller (orquestración bajo el patrón MVC). Esta arquitectura garantiza desacoplamiento, escalabilidad y estandarización en el consumo de datos, de modo que un cambio en la capa de programación o en el esquema de la base de datos impacta únicamente la capa correspondiente. Los resultados evidencian que la combinación de POO y patrones de diseño produce código mantenible y reutilizable, comparable a frameworks ORM comerciales como SQLAlchemy o Django ORM.

Palabras clave: ORM nativo, Repository, Domain Model, Unit of Work, CRUD, Python POO, MySQL Sakila, MVC, métricas descriptivas.

Introducción

La gestión eficiente de bases de datos relacionales es una competencia fundamental en el campo de la Ciencia de Datos. Los Sistemas de Gestión de Bases de Datos (SGBD) constituyen el componente central de prácticamente todo sistema de información moderno, encargándose de la persistencia, la integridad y la recuperación eficiente de los datos (Ramakrishnan & Gehrke, 2003). El acceso a estos datos mediante código de aplicación ha evolucionado desde consultas SQL directas hacia patrones arquitectónicos que abstraen la complejidad de la capa de persistencia, facilitando la mantenibilidad y la escalabilidad del software (Fowler, 2002).

En este contexto, las Soluciones de Mapeo Objeto-Relacional (ORM, por sus siglas en inglés) han surgido como el mecanismo predominante para mediar entre el paradigma orientado a objetos y el modelo relacional, abordando el conocido problema de la *discordancia de impedancia* (impedance mismatch). Tras dos décadas de evolución, los ORM se han consolidado en todas las plataformas de desarrollo populares, aunque persisten desafíos relacionados con la nomenclatura inconsistente entre soluciones y las implicaciones de diseño que cada estrategia de mapeo conlleva (Torres et al., 2017).

El presente caso práctico utiliza la base de datos Sakila, distribuida por Oracle como base de datos de demostración para MySQL. Sakila modela el sistema de información de una cadena de videotiendas e incluye entidades como Film, Inventory, Customer, Rental y Payment, lo que la convierte en un escenario idóneo para practicar consultas analíticas y operaciones de mantenimiento de datos (Oracle Corporation, 2023).

El objetivo principal es construir, en dos fases, un sistema que evolucione desde un CRUD funcional con consultas directas hacia una arquitectura ORM propia organizada bajo el patrón Modelo-Vista-Controlador (MVC), el cual promueve la separación de responsabilidades entre la representación de datos, la lógica de presentación y el control de las interacciones del sistema (Leff & Rayfield, 2001). Esta arquitectura se sustenta en los principios de encapsulamiento, herencia y polimorfismo propios de la Programación Orientada a Objetos (POO).

Fase I: CRUD Básico y Consultas SQL

Configuración del Entorno

Se instaló MySQL Community Edition 8.0 junto con MySQL Workbench 8.0 como IDE de administración visual. La base de datos Sakila se importó mediante el wizard de importación del Workbench, ejecutando los scripts sakila-schema.sql y sakila-data.sql provistos por Oracle (Oracle Corporation, 2023). La conexión desde Python se establece usando el paquete mysql-connector-python con las credenciales configuradas en el archivo config.py.

Consultas SQL (Q1–Q10)

Se diseñaron diez consultas analíticas que cubren distintas dimensiones del negocio de videotienda, desde distribución de catálogo hasta métricas de ingresos por tienda:

#	Consulta	Descripción
Q1	Películas por categoría	Cuenta total de títulos agrupados por género, ordenados de mayor a menor
Q2	Top 10 más rentadas	Películas con mayor número de rentas históricas
Q3	Ciudades por país	Top 15 países con más ciudades registradas

#	Consulta	Descripción
Q4	Ingresos por tienda	Suma total de pagos agrupada por tienda, ciudad y país
Q5	Actores más activos	Top 10 actores por número de apariciones en películas
Q6	Clientes con mayor gasto	Top 10 clientes por monto acumulado de pagos
Q7	Inventario por tienda y categoría	Copias disponibles cruzando store con categoría
Q8	Clasificación y duración	Promedio, mínimo y máximo de duración por rating
Q9	Rentas activas	Registros sin fecha de devolución y días transcurridos
Q10	Idiomas disponibles	Conteo de películas por idioma de la tabla language

CRUD Básico en Python

Se implementaron tres módulos independientes (`crud_city.py`, `crud_country.py`, `crud_film.py`) siguiendo el principio de responsabilidad única. Cada módulo expone las cuatro operaciones CRUD mediante funciones nombradas según la convención `create_*`, `read_*`, `update_*`, `delete_*`.

Estructura de las funciones CRUD

```
# Ejemplo: crud_city.py
def create_city(city_name: str, country_id: int) -> int: ...
def read_all_cities() -> list: ...
def read_city_by_id(city_id: int) -> dict: ...
def update_city(city_id: int, new_name: str = None, ...) -> None: ...
def delete_city(city_id: int) -> None: ...
```

Importación y exportación CSV / JSON

Cada módulo incluye cuatro funciones de transferencia de datos: `export_to_csv`, `import_from_csv`, `export_to_json` e `import_from_json`. Los archivos generados se almacenan en la carpeta `data/` del proyecto.

Métricas descriptivas

El módulo `metrics.py` calcula las cinco métricas fundamentales sobre columnas numéricas de Sakila (`rental_rate`, `length`, `amount`) sin usar librerías externas, aplicando las fórmulas estadísticas directamente en Python:

Métrica	Fórmula	Columnas aplicadas
Media	$\Sigma x / n$	<code>rental_rate</code> , <code>length</code> , <code>amount</code>
Rango	$\max(x) - \min(x)$	<code>rental_rate</code> , <code>length</code> , <code>replacement_cost</code>
Varianza muestral	$\Sigma(x-\mu)^2 / (n-1)$	<code>rental_rate</code> , <code>length</code>
Desviación estándar	$\sqrt{\text{varianza}}$	<code>rental_rate</code> , <code>amount</code>
Covarianza	$\Sigma(x-\mu_x)(y-\mu_y)/(n-1)$	<code>rental_rate</code> vs <code>replacement_cost</code>

Mejora de Integridad: UNIQUE Constraints

La integridad de los datos constituye uno de los pilares fundamentales de los sistemas de bases de datos relacionales, y se garantiza tanto por restricciones a nivel de esquema como por reglas semánticas definidas por el diseñador (Ramakrishnan & Gehrke, 2003). Con el propósito de fortalecer dicha integridad, se aplicaron restricciones UNIQUE sobre las siguientes tablas mediante scripts DDL ejecutados en Workbench:

Tabla	Constraint	Campos cubiertos
<code>country</code>	<code>uq_country_name</code>	<code>country</code>
<code>city</code>	<code>uq_city_country</code>	<code>city</code> , <code>country_id</code>
<code>film</code>	<code>uq_film_title_year</code>	<code>title</code> , <code>release_year</code>
<code>language</code>	<code>uq_language_name</code>	<code>name</code>
<code>category</code>	<code>uq_category_name</code>	<code>name</code>
<code>actor</code>	<code>uq_actor_fullname</code>	<code>first_name</code> , <code>last_name</code>

Posterior a la aplicación de los constraints se ejecutaron diez consultas adicionales (Q11–Q20) orientadas a validar la integridad de los datos, identificar duplicados remanentes, analizar el rendimiento del personal y detectar películas sin inventario.

Fase II: ORM Nativo en POO

Arquitectura de la Solución

El ORM implementado adopta el patrón Modelo-Vista-Controlador (MVC) adaptado al contexto de acceso a datos. La arquitectura se compone de cuatro capas que se comunican en cascada, siguiendo los patrones estructurales identificados en la literatura sobre soluciones ORM, como Data Mapper, Active Record y Repository (Torres et al., 2017). Este diseño es conceptualmente equivalente al implementado en frameworks de amplia adopción como SQLAlchemy (Bayer, 2012):

Capa	Archivo	Responsabilidad
DbContext	db_context.py	Gestión Singleton de la conexión MySQL. Provee <code>execute_query</code> y <code>execute_non_query</code> como métodos genéricos.
Entity	entities.py	Espejo Python de cada tabla. Atributos = campos de la tabla. Métodos: <code>from_dict()</code> , <code>to_dict()</code> .
Repository	repositories.py	Colección <code>List<Entity></code> que encapsula el acceso a datos. Implementa los métodos CRUD orquestando Entity + DbContext.
Controller	controllers.py	Capa de entrada MVC. Valida inputs, llama al Repository y retorna respuestas estándar <code>{status, message, data}</code> .

DbContext

El DbContext implementa el patrón Singleton para garantizar que exista una única instancia de conexión durante el ciclo de vida de la aplicación. Expone métodos genéricos que abstraen la escritura de código de conexión repetitivo en cada operación CRUD.

```
class DbContext:
    _instance = None # Singleton
    def execute_query(self, sql, params=()) -> list:
        # Ejecuta SELECT, retorna List[dict]
    def execute_non_query(self, sql, params=()) -> int:
        # Ejecuta INSERT/UPDATE/DELETE, retorna lastrowid
```

Entities

Cada Entity es una clase Python que replica fielmente los campos de su tabla correspondiente en la base de datos. Todas heredan de BaseEntity, que provee los métodos to_dict() y __repr__. El método de clase from_dict() permite instanciar una Entity a partir de un diccionario retornado por el cursor de MySQL.

```
class FilmEntity(BaseEntity):
    def __init__(self, film_id, title, description, release_year,
                  language_id, rental_duration, rental_rate,
                  length, replacement_cost, rating, last_update): ...
    @classmethod
    def from_dict(cls, d: dict) -> 'FilmEntity': ...
    def to_dict(self) -> dict: ...
```

Repositories (List<Entity>)

Cada Repository encapsula la lógica de acceso a datos para una entidad específica, presentándose ante el resto de la aplicación como una colección en memoria. Esta abstracción es una variante del patrón Repository, en el cual un objeto media entre la capa de dominio y la capa de mapeo de datos, actuando como una colección de objetos del dominio (Fowler, 2002). Cada Repository mantiene una lista interna de entidades (self._items: list) y expone los métodos CRUD que interactúan con la base de datos a través

del DbContext, garantizando que la capa de presentación nunca manipule SQL directamente.

```
class FilmRepository(BaseRepository):
    def load_all(self, limit=200) -> list[FilmEntity]: ...
    def get_by_id(self, film_id) -> FilmEntity: ...
    def search_by_title(self, keyword) -> list[FilmEntity]: ...
    def insert(self, entity: FilmEntity) -> int: ...
    def update(self, entity: FilmEntity) -> bool: ...
    def delete(self, film_id) -> bool: ...
```

Controllers (MVC)

Los Controllers actúan como la capa de entrada del sistema MVC, recibiendo las solicitudes externas y orquestando la interacción entre la vista y el modelo. Esta función de mediación es la responsabilidad clásica del Controller en el patrón original (Leff & Rayfield, 2001). Cada método del Controller valida las entradas, delega la operación al Repository correspondiente y retorna una respuesta estándar en formato diccionario con tres campos: status (success/error), message y data.

```
class FilmController(BaseController):
    def get_all(self, limit=50) -> dict:
        films = self.repository.load_all(limit=limit)
        return self.ok(data=[f.to_dict() for f in films],
                        message=f'{len(films)} películas cargadas.')
    def create(self, title, description='', ...) -> dict:
        if not title: return self.error('El título no puede estar vacío.')
        entity = FilmEntity(title=title, ...)
        new_id = self.repository.insert(entity)
        return self.ok(data={'film_id': new_id}, message='Creado.')
```

Flujo Completo de una Operación

El siguiente diagrama ilustra el flujo de una operación CREATE Film a través de las cuatro capas de la arquitectura ORM:

Paso	Capa	Acción
1	Controller	Recibe parámetros, valida que el título no sea vacío
2	Controller	Instancia un FilmEntity con los datos recibidos
3	Repository	Llama a FilmRepository.insert(entity)
4	DbContext	Ejecuta el SQL INSERT, retorna film_id generado
5	Repository	Agrega la entity a self._items (lista en memoria)
6	Controller	Retorna {status:'success', data:{film_id:X}, message:'Creado.'}

Resultados

La ejecución del script de demo (fase2/main.py) con la base de datos Sakila local confirma el funcionamiento de la arquitectura en las cuatro entidades implementadas: Country, City, Film y Customer. Las operaciones CRUD se completan satisfactoriamente y el sistema de respuesta estándar permite distinguir operaciones exitosas de errores de forma uniforme.

Las métricas descriptivas calculadas sobre la tabla film arrojan los siguientes resultados representativos: la tarifa de renta (rental_rate) presenta una media de \$2.98 con desviación estándar de \$1.65 y rango de \$4.00 (desde \$0.99 hasta \$4.99). La duración promedio de las películas es de 115.27 minutos, con una desviación estándar de 40.43 minutos. La covarianza entre rental_rate y replacement_cost es positiva, lo que sugiere una asociación moderada entre ambas variables comerciales.

Estructura del Proyecto

Archivo	Tipo	Propósito
config.py	Configuración	Credenciales de conexión a MySQL
test_connection.py	Utilidad	Verifica la conexión y lista tablas

Archivo	Tipo	Propósito
fase1/queries.sql	SQL	10 consultas analíticas Q1–Q10
fase1/unique_constraints.sql	DDL + SQL	Constraints UNIQUE + 10 consultas Q11–Q20
fase1/crud_city.py	Python	CRUD + CSV/JSON para tabla city
fase1/crud_country.py	Python	CRUD + CSV/JSON para tabla country
fase1/crud_film.py	Python	CRUD + CSV/JSON para tabla film
fase1/metrics.py	Python	Métricas descriptivas sobre Sakila
fase2/db_context.py	ORM	DbContext Singleton
fase2/entities.py	ORM	Entities: Country, City, Film, Customer
fase2/repositories.py	ORM	Repositories con List<Entity>
fase2/controllers.py	ORM	Controllers patrón MVC
fase2/main.py	Demo	Prueba completa del ORM

Conclusiones y Recomendaciones

La implementación de un ORM nativo en Python sobre la base de datos Sakila demuestra que los principios de la POO son suficientes para construir una capa de abstracción de datos robusta y mantenible sin recurrir a frameworks de terceros. La separación en cuatro capas (DbContext, Entity, Repository, Controller) garantiza que cada componente tenga una responsabilidad única y bien definida, facilitando la extensión del sistema ante nuevos requerimientos.

El patrón Singleton en DbContext evita la proliferación de conexiones abiertas, un problema frecuente en sistemas de acceso a datos no estructurados. La respuesta estándar de los Controllers unifica el contrato de comunicación entre capas, simplificando la integración con interfaces de usuario o APIs REST futuras.

Como recomendaciones para trabajo futuro se propone: (1) integrar un sistema de connection pooling para entornos de alta concurrencia, (2) extender las métricas descriptivas con visualizaciones mediante Matplotlib, (3) exponer los Controllers como endpoints REST usando Flask o FastAPI, y (4) añadir pruebas unitarias con pytest para validar el comportamiento de cada capa de forma aislada.

Referencias

- Bayer, M. (2012). SQLAlchemy. En A. Brown & G. Wilson (Eds.), *The Architecture of Open Source Applications* (Vol. 2). aosabook.org.
<https://aosabook.org/en/sqlalchemy.html>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Leff, A., & Rayfield, J. T. (2001). Web-application development using the Model/View/Controller design pattern. En *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference (EDOC 2001)* (pp. 118–127). IEEE. <https://doi.org/10.1109/EDOC.2001.950428>
- Oracle Corporation. (2023). *Sakila sample database*. MySQL Developer Zone.
<https://dev.mysql.com/doc/sakila/en/>
- Ramakrishnan, R., & Gehrke, J. (2003). *Database management systems* (3ra ed.). McGraw-Hill.
- Torres, A., Galante, R., Pimenta, M. S., & Martins, A. J. B. (2017). Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design. *Information and Software Technology*, 82, 1–18.
<https://doi.org/10.1016/j.infsof.2016.09.009>

Anexo A: Repositorio y Código Fuente

El código completo del proyecto se encuentra disponible en el repositorio de GitHub del equipo:

https://github.com/mudstart/caso_practico_2

El repositorio incluye todos los scripts SQL, módulos Python de Fase I y Fase II, archivos de datos de ejemplo en CSV y JSON, y el presente documento en formato .docx.

Anexo B: Instrucciones de Ejecución

1. Instalar dependencia

```
pip install mysql-connector-python
```

2. Verificar conexión a Sakila

```
python test_connection.py
```

3. Ejecutar consultas SQL en Workbench

Abrir: fase1/queries.sql

Abrir: fase1/unique_constraints.sql

4. Probar CRUD básico (Fase I)

```
python fase1/crud_city.py
```

```
python fase1/crud_film.py
```

```
python fase1/metrics.py
```

5. Demo ORM completo (Fase II)

```
python fase2/main.py
```